

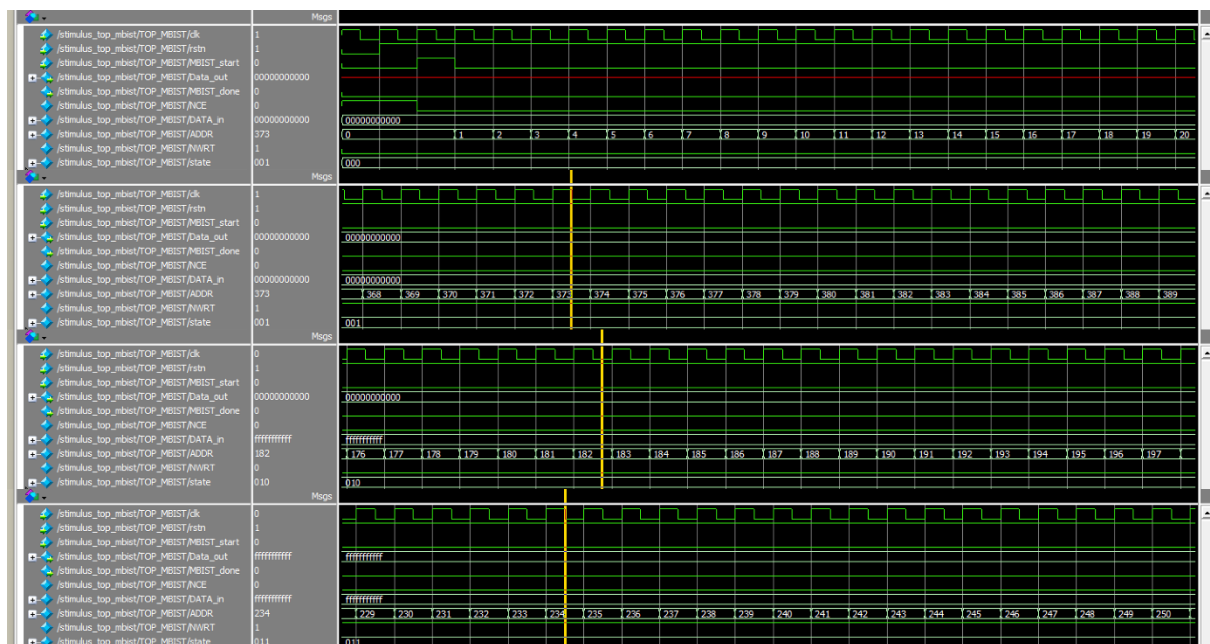
VLSI Practice 06 Report

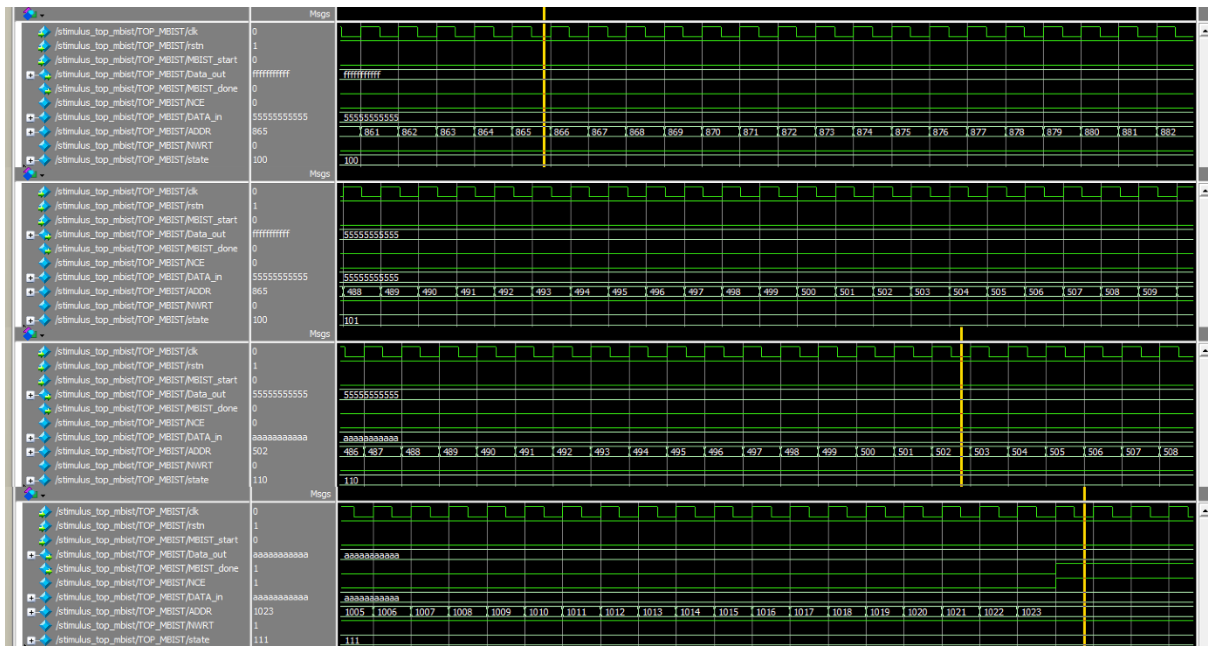
2022171005 권민찬

1. MBIST

MBIST는 메모리가 정상 작동하는지 검증하기 위해 사용하는 모듈이다. 이번 실험에서는 1024 x 44-bit memory를 검증하기 위한 모듈을 만들었다. 이 메모리는 1024개의 주소를 가지므로, 주소를 표현하기 위해 10비트가 필요하다. 모든 주소에 대해서 네 가지 데이터 패턴(44'h000~, 44'hffff~, 44'h5555~, 44'haaaa~)을 쓰고, 다시 네 가지 데이터 패턴을 읽기 위해 2+1개의 상태(state) 비트가 필요하다. 따라서 총 13비트를 가지는 카운터를 이용해 10개의 LSB는 주소비트, 그 다음 비트는 NWRT, 그리고 2개의 MSB는 데이터 패턴을 나타내도록 설계했다. 최종적으로 8가지 종류의 테스트가 종료되면 MBIST_done 신호가 1을 출력하도록 했다.

i. Timing Simulation

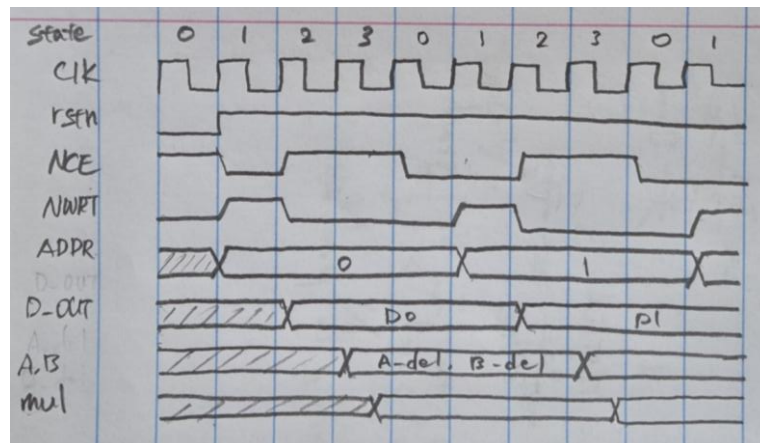




가장 위의 그림부터 state가 0일 때, 1일 때, ... 7일 때의 타이밍 다이어그램이다. State의 가장 하위 비트가 NWRT 신호이므로 메모리에 READ를 인가했을 때(즉, 하위 비트가 1일 때) 출력되는 Data_out이 이전 상태에서 메모리에 썼던 DATA_in과 같은 것을 볼 수 있다. 가장 처음 state가 0일 때는 아직 메모리에 값을 쓰지도 않았고, 읽고 있지도 않기 때문에 Data_out에 undetermined가 출력되는 것을 볼 수 있다. 모든 상태에서 주소는 0 부터 1023까지 증가하며 읽기와 쓰기를 반복하고 있고, 마지막 state에서 테스트가 완료 된 후에는 MBIST_done이 1이 되고, negative enable인 NCE도 1이 되어 메모리가 비활성화되는 것을 볼 수 있다.

2. Memory Control with 22-bit Multiplier

i. Timing Diagram



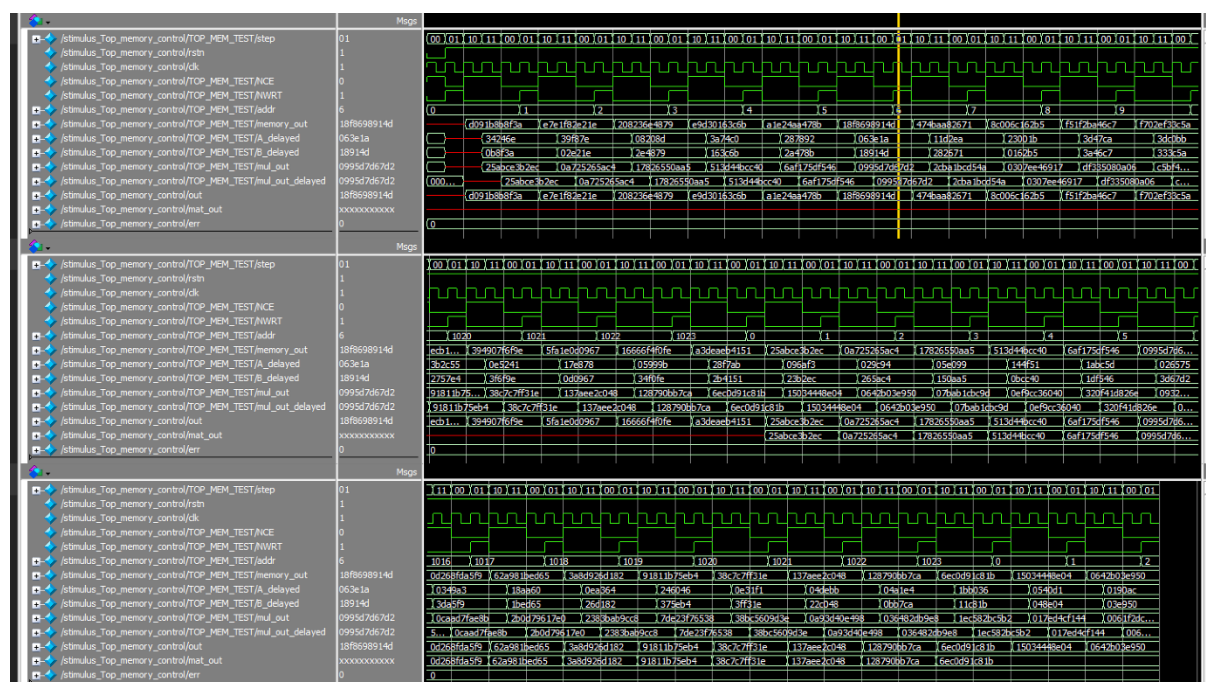
위의 그림은 메모리 컨트롤러의 초반 동작을 나타낸 timing diagram이다. 가장 처음 rstn

이 한 클럭 사이클동안 인가되는데, 이때 state=0, NCE=1, NWRT=0으로 초기화된다. 그 이후 rstn=1이 되면서 모듈이 동작을 시작하게 된다. 이 설계에서 어려웠던 부분은 NCE와 NWRT가 클럭의 파형과 다르다는 것이다. NCE의 경우 동작 시작 이후 0-1-1-0의 파형을 가지고 있고, NWRT는 1-0-0-0의 파형을 가지고 있다. 따라서 단순한 always @(posedge clk)을 이용해서는 두 신호를 출력하기 어려운 문제가 있었다. 이를 해결하기 위해 step 카운터를 도입했다. Rstn=0일 때는 step가 0으로 초기화되고, 동작 시작 때는 1부터 다시 0까지 카운트를 하게 된다.

Previous step	NCE	NWRT
00	0	1
01	1	0
10	1	0
11	0	0

위의 표는 이전 step값과 NCE, NWRT를 Truth table로 나타낸 것이다. NCE와 NWRT를 이전 step을 이용해 표현하면 $NCE \leq \text{step}[0] \wedge \text{step}[1]$, $NWRT \leq \sim(\text{step}[0] \vee \text{step}[1])$ 이다. 이 Boolean algebra를 이용해 always @(posedge clk) 블록 안에서 NCE와 NWRT를 synchronous하게 다룰 수 있게 됐다. 또한 파형을 봤을 때 addr 값은 이전 NCE와 NWRT가 모두 0일 때 바뀌게 되므로 이 관계를 이용하면 addr도 적절한 시점에 증가시킬 수 있다.

ii. Timing Simulation



위의 타이밍 시뮬레이션 결과는 손으로 그린 타이밍 다이어그램과 같은 형태의 파형이 출력되는 것을 볼 수 있다. 초반 rstn과 동작 시작 후의 파형이 일치하는 것을 볼 수 있고, 그 이후에도 4개의 클럭 사이클마다 새로운 결과가 출력되는 것을 볼 수 있다. 곱셈기 전후에 레지스터를 추가하여 곱셈기가 안정적으로 operand를 입력받고, 출력값이 메모리에 안정적으로 저장될 수 있게 하였다. 레지스터가 추가되어 생기는 클럭 사이클 지연도 확인할 수 있다. 최종적으로 모든 메모리를 읽었을 때 하단의 err=0인 것을 볼 수 있다.

3. Discussion

두번째 실험에서는 rstn, NCE, NWRT, addr 등의 신호를 언제 인가해야 하는지에 대해 고민해야 할 부분이 많았다. 이런 문제를 해결하기 위해 step 카운터를 도입했다. 이 step 카운터와 NCE, NWRT의 관계에 대해서 생각해보면, NCE와 NWRT는 이전 step에 따라 값이 바뀌기 때문에 FSM으로 생각해도 문제가 없다. 즉 어떤 신호값을 다른 신호들 사이의 관계로 생각하기보다는, state 개념을 도입해서 각 state마다 출력해야 할 신호값을 정의해주는 것이 설계하기 더 편할 것이다. 특히 신호가 점점 더 많아지고 디자인이 더 커지게 되면 FSM을 설계하는 방식이 오류를 줄이고 디버깅하기 쉬울 것이다.

현재 디자인에서는 메모리에서 읽기를 수행한 후 두 개의 클럭 사이클에서는 메모리를 사용하지 않는 곱셈 연산이 진행된다. 따라서 곱셈이 진행되는 동안 병렬적으로 이전 메모리 주소에서 이전 단계의 곱셈 결과를 읽는 방식으로 속도 향상을 꾀할 수 있을 것이다.